

Programmation web : Rapport



Réalisé par:
Serigne Fall
Abdallah Remmide

Enseignant:
Morgane Vidal

Année universitaire 2025/2026

Sommaire

Introduction	2
--------------	---

Exercice 1 : Le client web

2.1 Architecture et choix	3
2.2 Guide d'utilisation de l'application	3
2.3 Liste des endpoints	4

Exercice 2 : La guerre des chats

3.1 Architecture du backend et de la base de données	4
3.2 Architecture du client web et sécurité	5
3.3 Guide d'utilisation et identifiants	5
3.4 Liste des endpoints	6

Conclusion	7
------------	---

1. Introduction

Ce rapport présente le devoir maison de programmation web, qui contient deux exercices développés à la suite des TP de ce semestre. L'objectif global de ce projet était de démontrer notre maîtrise de la communication entre un client et un serveur, en abordant les deux aspects du développement web.

Tout d'abord, l'exercice 1 nous a mis dans la position de consommateur d'une API publique. Il s'agissait de construire une interface frontend capable d'interroger cette source de données et d'en formater les résultats. Puis, l'exercice 2 nous a placés dans le rôle de créateur d'une API RESTful avec FastAPI. Cette application, "La guerre des chats", nécessitait la conception d'une base de données locale et la mise en place d'un système d'authentification. Ce rapport détaille l'architecture et le fonctionnement de ces deux applications.

2. Exercice 1 : Le client web

2.1. Architecture et choix

Pour ce client web, nous avons choisi de créer une Single Page Application en utilisant uniquement du HTML, CSS et du JavaScript. L'idée était de garder un projet léger, sans s'encombrer d'un framework complexe. Toute la structure est dans notre fichier index.html, et la logique des requêtes se trouve dans script.js.

Pour naviguer entre les onglets sans avoir à recharger la page, notre JavaScript manipule directement le DOM. Il ajoute ou retire une classe CSS pour cacher les sections inactives. Pour le design, on a utilisé des variables CSS pour respecter les couleurs de Pokémon. L'interface est responsive, et on a ajouté un petit détail visuel avec une marge négative sur le logo pour qu'il déborde sur la barre de navigation.

Enfin, niveau performances, l'API étant énorme, tout charger au démarrage aurait fait ramer la page. Nous avons donc mis en place un chargement à la demande. Le code lance la requête vers le serveur uniquement quand l'utilisateur ouvre un menu déroulant. Si on ne va pas dans l'onglet des baies, on ne télécharge rien. Pour optimiser le tout, on a aussi stocké nos éléments HTML dans des variables au tout début du script pour éviter au navigateur de les chercher à chaque clic.

2.2. Guide d'utilisation de l'application

L'application est intuitive et se divise en trois onglets accessibles depuis la barre de navigation principale. Au lancement du fichier HTML, l'utilisateur arrive sur l'onglet de recherche. Dans cet espace, une barre de saisie permet d'entrer soit le numéro d'identifiant, soit le nom du Pokémon. Une fois la recherche validée, l'application génère une carte détaillant les informations du Pokemon. On y retrouve l'image, le poids et la taille que nous avons convertis en kilogrammes et en mètres, ainsi que la description. Un lecteur audio est également généré pour permettre d'écouter le cri du Pokemon. En cas de faute de frappe, notre code affiche un message d'avertissement à l'utilisateur au lieu de faire planter la page.

Le deuxième onglet propose une navigation par génération. L'utilisateur peut sélectionner l'une des générations dans un menu déroulant. L'application construit alors une grille affichant tous les individus appartenant à cette époque et triés par numéro grâce à un algorithme de traitement des URL. De plus, si l'utilisateur clique sur l'une des cases de cette grille, l'application bascule automatiquement sur l'onglet de recherche et charge la fiche complète du Pokemon correspondant.

Enfin, le troisième onglet est dédié aux baies. Le principe de navigation reste la même avec un menu de sélection listant les éléments disponibles. Lorsqu'un choix est effectué, notre application interroge plusieurs endpoints de l'API de manière asynchrone pour reconstituer une fiche qui contient le nom, le niveau de fermeté, l'image de l'objet et une description.

2.3. Liste des endpoints

Voici la liste des endpoints que notre application web utilise pour fonctionner :

- **GET `https://pokeapi.co/api/v2/pokemon/{id_ou_nom}`** : endpoint principal de recherche pour extraire les statistiques, les sprites visuels et les fichiers audio.
- **GET `https://pokeapi.co/api/v2/pokemon-species/{id}`** : endpoint secondaire pour chercher les entrées textuelles.
- **GET `https://pokeapi.co/api/v2/generation`** : endpoint qui permet de récupérer la liste des générations pour le menu de tri.
- **GET `https://pokeapi.co/api/v2/generation/{nom_generation}`** : endpoint qui nous fournit la liste des créatures associées à une période spécifique.
- **GET `https://pokeapi.co/api/v2/berry?limit=100`** : endpoint qui liste l'inventaire de base pour construire la sélection de l'utilisateur.
- **GET `https://pokeapi.co/api/v2/berry/{nom_baie}`** : endpoint qui fournit les données de l'objet comme la fermeté.
- **GET `https://pokeapi.co/api/v2/item/{nom_item}`** : endpoint pour pouvoir récupérer la description qui n'est pas fourni par la route des baies.

3. Exercice 2 : La guerre des chats

3.1. Architecture du backend et de la base de données

Le développement de cette seconde application s'appuie sur le framework FastAPI en Python. Contrairement aux premiers TP de ce projet où les données étaient statiques dans un fichier JSON ou créées artificiellement au démarrage du serveur, nous avons mis en place une base de données SQLite. Cela nous a permis de définir des classes Python qui se traduisent automatiquement en tables sécurisées.

Le cœur de ce travail a consisté à modéliser les relations entre les différentes entités. Nous avons conçu quatre tables distinctes : les utilisateurs, les armées, les chats et les armes. Pour lier ces informations, nous avons configuré des clés étrangères. Ainsi, chaque armée possède l'identifiant unique de son propriétaire dans la base de données, et chaque chat enregistré stocke l'identifiant de l'armée à laquelle il a été affecté et l'identifiant de son équipement. Cette structure relationnelle garantit l'intégrité des données et permet de s'assurer qu'un soldat ne se retrouve jamais seul ou rattaché à deux camps en même temps.

Nous avons intégré la bibliothèque Passlib associée à Bcrypt pour hacher les données d'authentification avant de les écrire dans la base SQLite. Pour la gestion des sessions, nous avons développé un système complet de jetons JSON Web Token. Lorsqu'un utilisateur prouve son identité, le serveur crée un jeton cryptographique signé numériquement et valable 30 minutes.

3.2. Architecture du client web et sécurité

Pour accompagner cette API, nous avons développé une interface web dans un fichier HTML. Nous avons donc paramétré les filtres de requêtes de notre application FastAPI pour autoriser les échanges avec notre frontend.

L'interface web repose sur un système d'états gérés en JavaScript. Tant que l'utilisateur n'est pas authentifié, seules les données publiques du serveur et le formulaire de connexion sont accessibles. Lors de la soumission du formulaire, le JavaScript encode les identifiants et récupère le jeton JWT renvoyé par le serveur. Ce jeton est alors conservé dans une variable globale côté client. Nous avons aussi configuré nos fonctions asynchrones pour qu'elles injectent systématiquement ce jeton sécurisé dans les en-têtes HTTP de chaque requête de modification, sous le format Bearer.

3.3. Guide d'utilisation et identifiants

L'interface utilisateur de la guerre des chats s'organise autour d'un tableau de bord dynamique. Dans la partie inférieure de la page se trouve la zone publique. N'importe quel visiteur peut la consulter en temps réel. On retrouve l'inventaire complet des bataillons ainsi que le registre de tous les chats : leur nom, leur race, leur âge et leur affectation.

Pour interagir avec le jeu, l'utilisateur doit utiliser la zone de connexion située en haut de page. Afin de faciliter la correction et les tests de l'application, nous avons pré-enregistré deux profils distincts dans la base de données. Vous pouvez vous connecter en utilisant les identifiants suivants :

identifiant: Abdallah

mot de passe: abdallah

identifiant: Serigne

mot de passe: serigne

Une fois connecté avec l'un de ces profils, le tableau de bord privé se déverrouille et masque le formulaire de connexion. L'utilisateur accède alors à son quartier général. Un premier formulaire lui permet de fonder ou de mettre à jour son armée en lui donnant un nom et une description. Un second formulaire est dédié au recrutement des soldats. L'utilisateur peut y saisir l'identité d'un nouveau chat. À la validation, la requête part vers le serveur, le nouveau combattant est enregistré dans la base de données SQLite et la liste publique s'actualise instantanément en bas de l'écran.

3.4. Liste des endpoints

L'application serveur FastAPI a été développée pour répondre aux routes suivantes, divisées selon leur niveau d'autorisation :

Endpoints d'Authentification:

- **POST /inscription** : point d'entrée pour la création d'un profil joueur, incluant le processus de hachage de sécurité.
- **POST /token** : point d'entrée critique vérifiant les identifiants et distribuant les jetons d'accès JWT pour la session.

Endpoints Sécurisés:

- **POST /mon-armee/** : route permettant au propriétaire du jeton d'insérer ou d'altérer les métadonnées de sa propre troupe.
- **POST /chats/recruter** : route autorisant la création d'une entité animale et son rattachement automatique à la clé étrangère du demandeur.
- **DELETE /chats/{chat_id}** : route de renvoi d'un soldat, nécessitant la validation formelle que la cible appartient bien au signataire de la requête.
- **PUT /chats/{chat_id}/equiper/{arme_id}** : route de gestion de l'équipement, intégrant les mêmes vérifications de propriété avant la modification de la base.

Endpoints Publics :

- **GET /armees/** : route interrogeant la table des armées pour exposer publiquement les forces constituées.
- **GET /chats/** : route extrayant l'ensemble des soldats du registre pour les afficher dans la zone neutre du client web.
- **GET /armes/** : route permettant de lister le catalogue des objets offensifs modélisés dans le système.
- **POST /armes/** : route ouverte laissée publique dans ce projet pour permettre l'enrichissement rapide de l'arsenal global.

4. Conclusion

L'élaboration de ce devoir maison nous a permis de faire la synthèse d'un grand nombre de concepts techniques abordés au fil du semestre pendant les TP. L'exercice 1 nous a confrontés aux réalités de l'exploitation de données asynchrones via la fonction Fetch et à l'importance de l'expérience utilisateur, notamment en gérant le DOM et en prévenant les surcharges de requêtes inutiles.

Pour la conception de la guerre des chats, la mise en place de la base de données relationnelle locale a structuré notre compréhension de l'intégrité des données, mais aussi les défis de la cybersécurité. Enfin, réussir à faire dialoguer notre client JavaScript avec notre API sécurisée nous a démontré l'importance de maîtriser les concepts de CORS et de transmission d'en-têtes HTTP.